University of Sialkot

Semester Project

(Client-Side & Server-side Architecture using Vue 3+ Vite+ Express.js)

Submitted by: Sabeeha Naveed (23010101-131)
Muneeba (23010101-052)

Department: Software engineering

Section: Blue

Subject: Web Engineering

Project Title: Medicine Reminder App

Submitted to: Sir Museb

Abstract

The Medicine Reminder System is a web-based application developed to help users manage their daily medications efficiently. The system allows users to register, log in securely, add medicines, set reminder schedules, update medicine records, and manage their personal profiles. The frontend is developed using Vue.js, while the backend is built with Node.js and Express.js. MongoDB Atlas is used as the cloud database to securely store user and medicine information. The application follows a client-server architecture and communicates through RESTful APIs.

1. Introduction

The Medicine Reminder System is designed to assist users in remembering their medication schedules. Missing medicines can negatively affect a patient's health; therefore, this application provides a digital solution for managing medicine records. Users can register, log in, add medicines, edit medicine details, and manage their profiles through an easy-to-use interface.

2. Objectives

The main objectives of the project are:

- Develop a secure medicine reminder application.
- Allow user registration and login.
- Manage medicine records using CRUD operations.
- Store data securely in MongoDB Atlas.
- Implement REST APIs for frontend-backend communication.
- Build a responsive Single Page Application (SPA).
- Improve medication management.

3. Technologies Used

Frontend

- Vue.js 3
- Vite
- Vue Router
- components
- Props
- Emitters
- CSS

Backend

- Node.js
- Express.js
- MongoDB Atlas
- Mongoose
- JWT
- CORS
- Dotenv

4. Project Structure

The project is divided into two main folders:

4.1 Views

The **Views** folder contains all the main pages of the application.

- Splash.vue
- Login.vue
- Signup.vue
- Dashboard.vue
- AddMedicine.vue
- Schedule.vue
- Profile. Vue
- EditProfile.vue
- Settings.vue

4.2 Components

- Sidebar. Vue
- MedicineCard.vue
- ProfileCard.vue
- SettingItem.vue

4.3 Frontend Project Folder Structure and Axios API Configuration

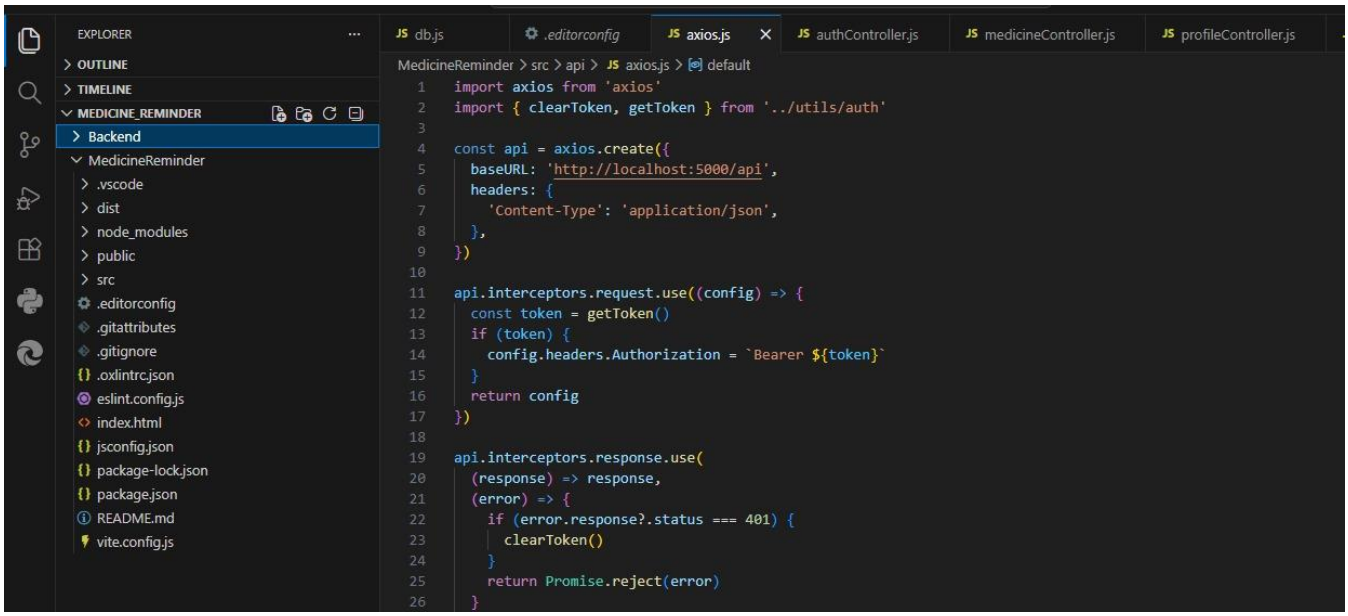


Figure 4.3: Frontend Project Folder Structure and Axios API Configuration

This figure shows the frontend folder structure developed using Vue.js. It also displays the axios.js file, where the Axios instance is configured with the base API URL, request interceptor for attaching the JWT token, and response interceptor for handling unauthorized requests.

4.4 Backend Project Folder Structure

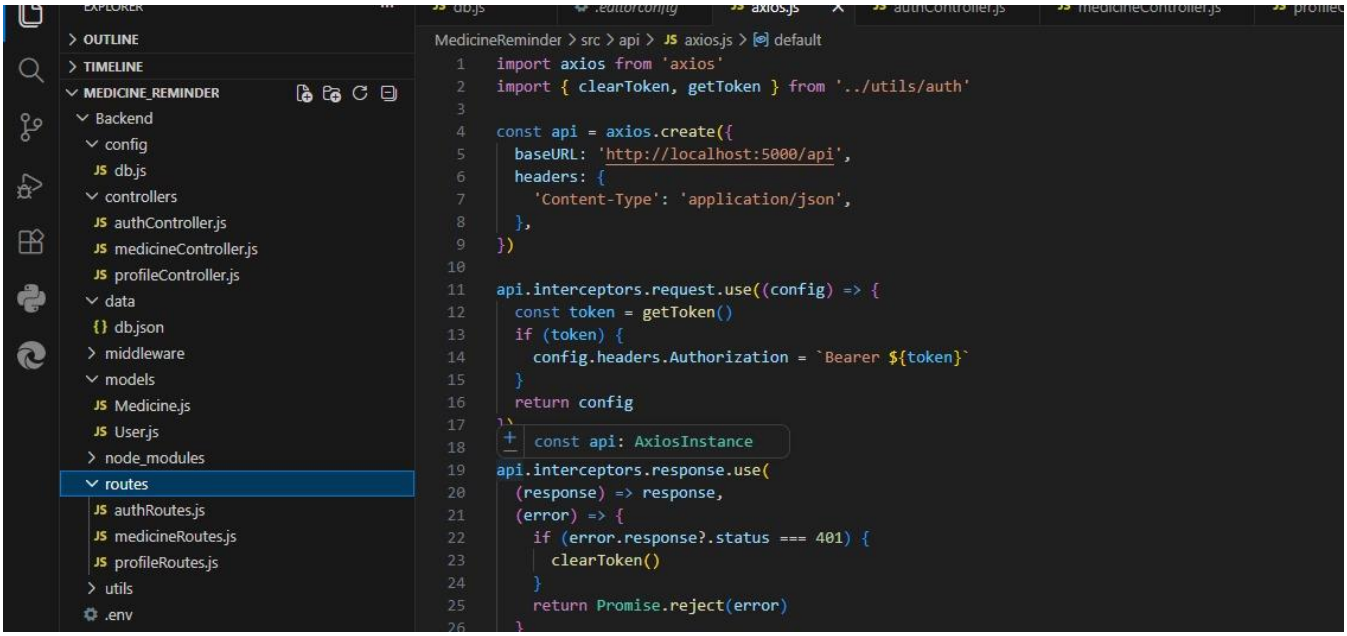


Figure 4.4 : Backend Project Folder Structure

This figure shows the backend project structure of the Medicine Reminder System. It includes folders such as config, controllers, models, routes, middleware, and utils, which organize the backend code following the MVC architecture.

5. Single Page Application (SPA)

The Medicine Reminder System is developed as a Single Page Application.

Advantages:

a) Faster navigation
b) No page reloads
c) Better user experience
d) Dynamic routing
e) Efficient data loading

6. Use Cases

a. User Registration
b. User Login
c. Forgot Password
d. View Dashboard
e. Add Medicine
f. View Medicines
g. Edit Medicine
h. Delete Medicine
i. View Profile
j. Edit Profile

k. Logout.

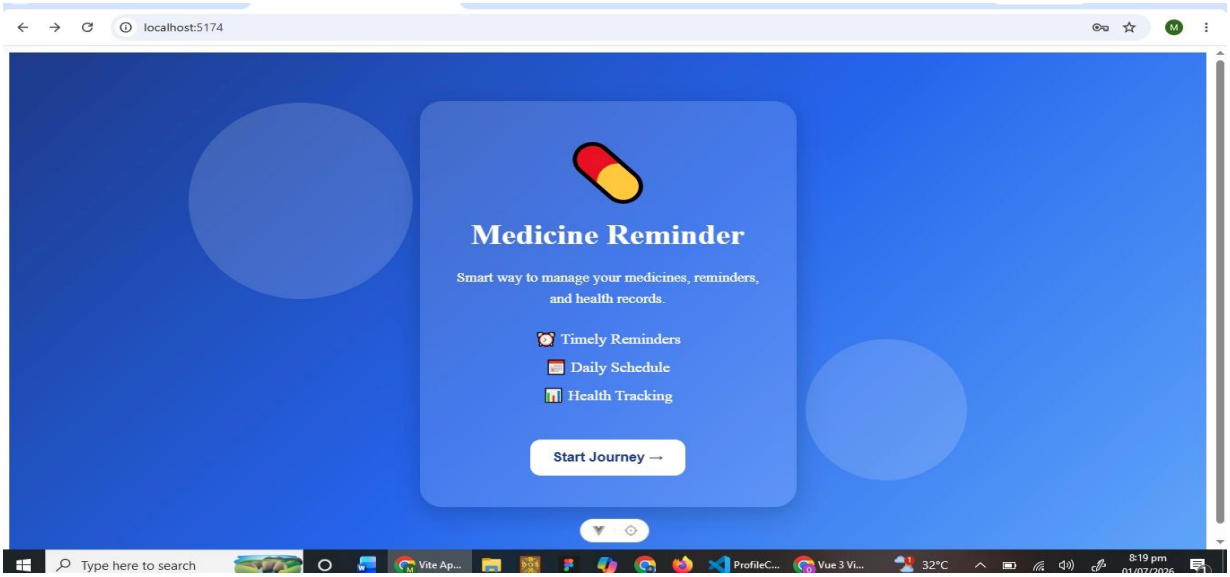
7. Pages Explanation

7.1 Splash Screen

The Splash Screen is the first page displayed when the application starts.

Purpose:

- Displays the app logo and name.
- Redirects the user to the Login or Signup page.



7.2 Login Page

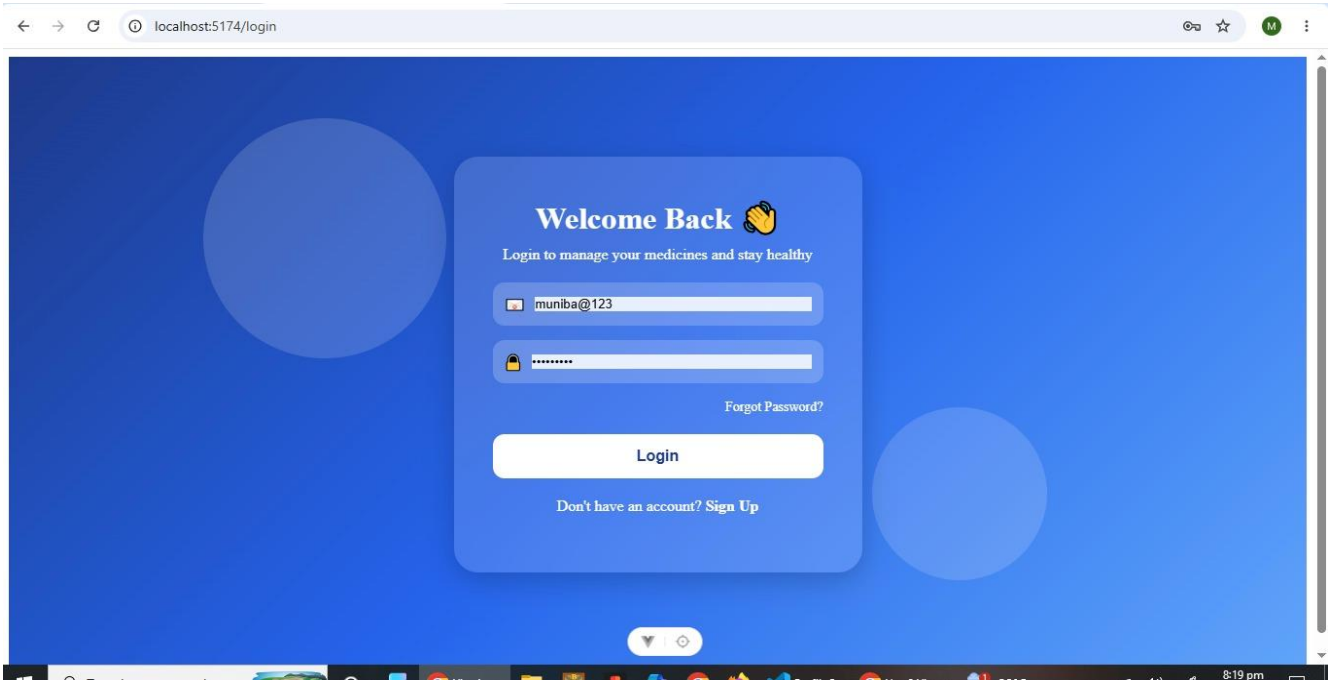
The Login page allows existing users to access the application.

Fields:

- Email
- Password

Button:

- Login



7.3 Signup Page

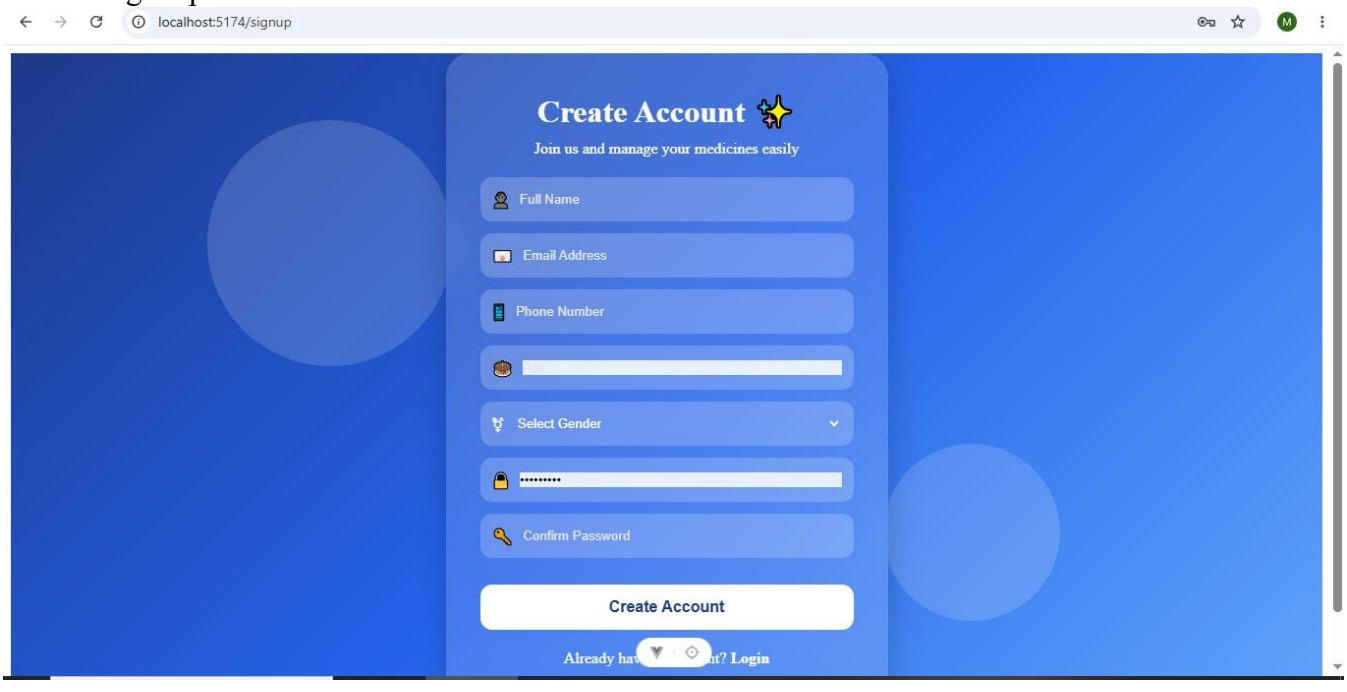
The Signup page allows new users to create an account.

Fields:

- Full Name
- Email
- Password
- gender

Button:

- Sign Up

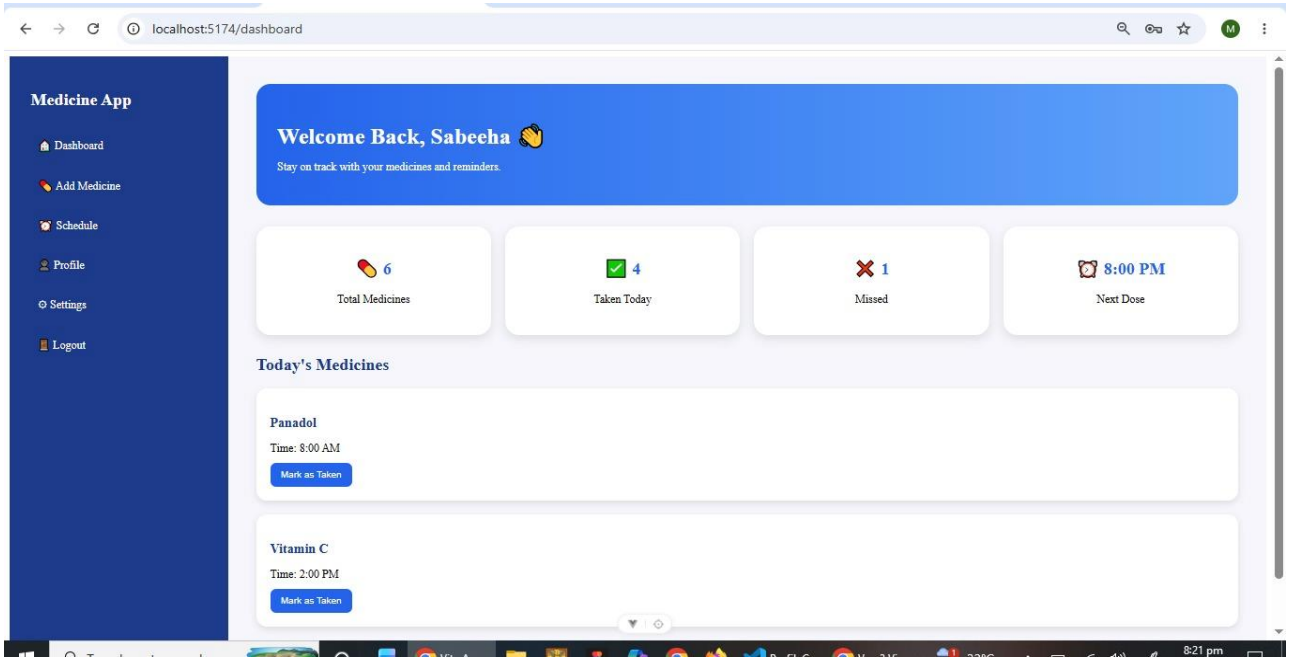


7.4 Dashboard

The Dashboard is the main home page after login.
It displays:

- Welcome message
- Total medicines
- Taken medicines
- Missed medicines
- Next medicine reminder

The **Medicine Card** component is used to display medicine information.

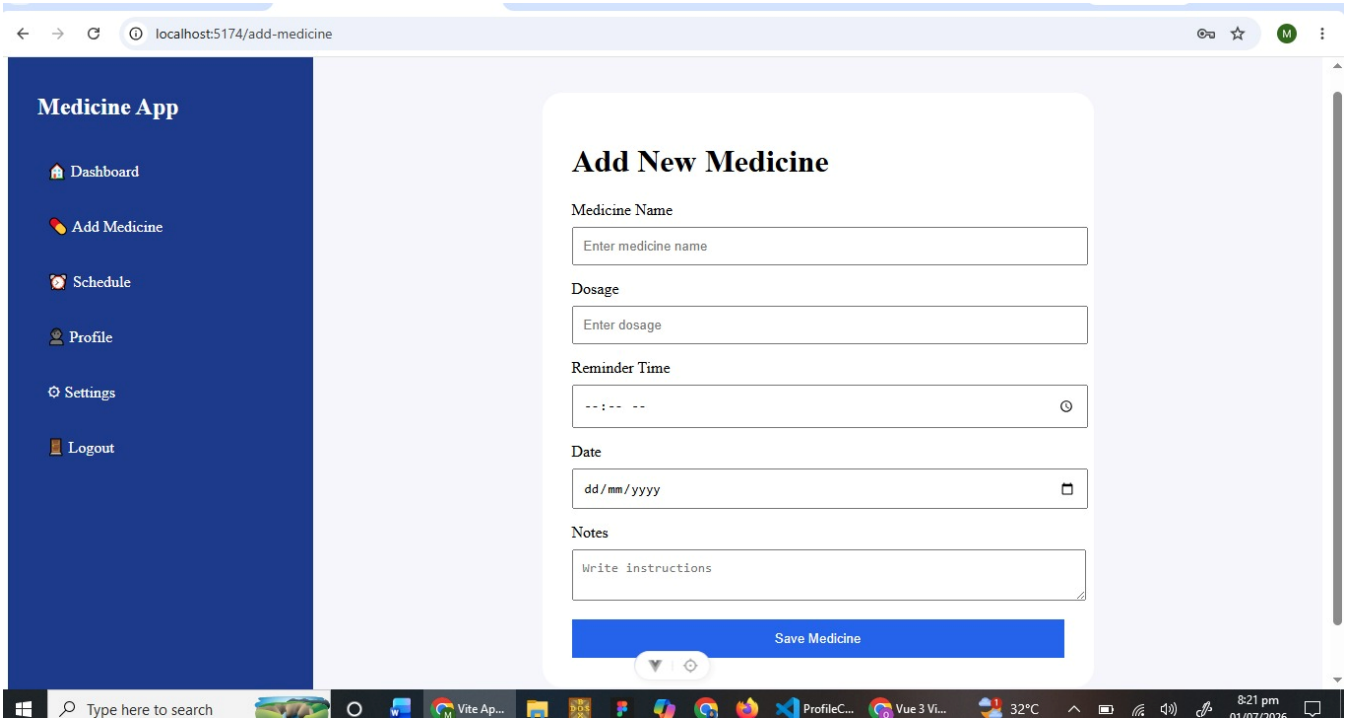


7.5 Add Medicine

This page allows users to add a new medicine.
Fields:

- Medicine Name
- Dose
- Reminder Time
- Date

Users can save the medicine for future reminders.



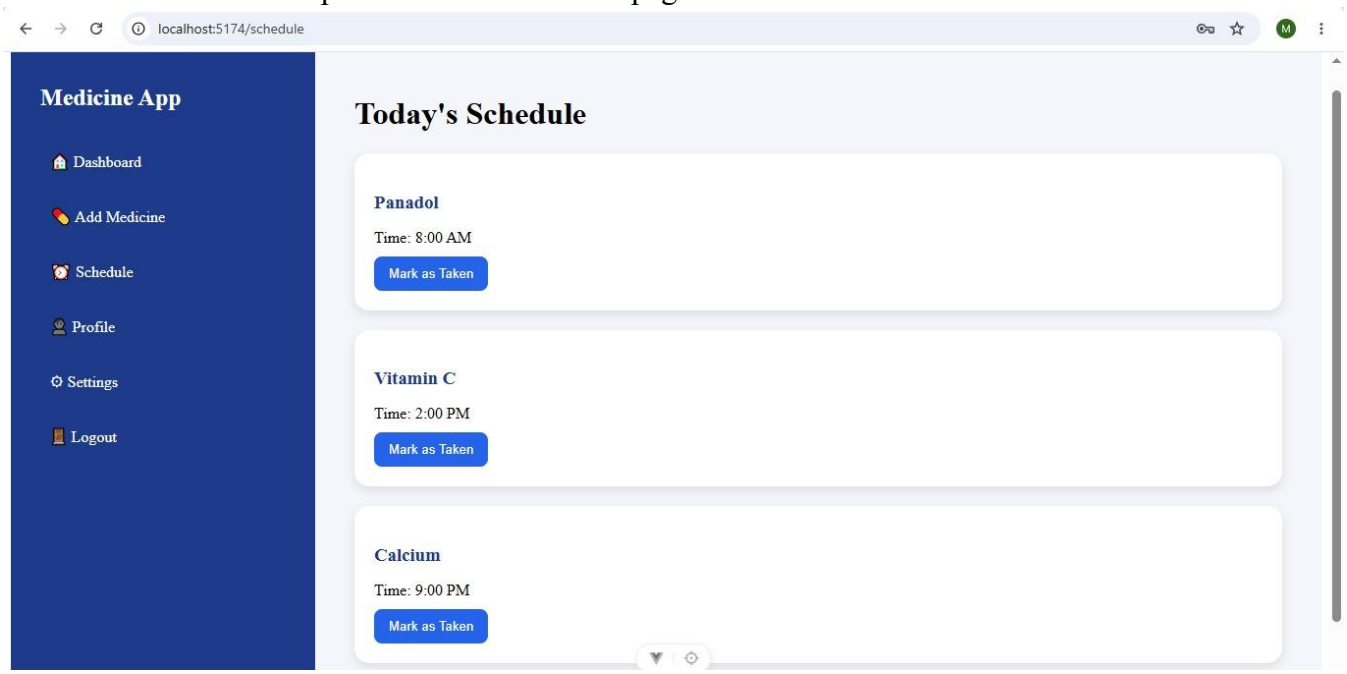
7.6 Schedule Page

The Schedule page displays all daily medicine reminders.

Users can:

- View medicine details
- Mark a medicine has taken

The **MedicineCard** component is reused on this page.



7.7 History Page

The History page keeps a record of previous medicines.
It shows:

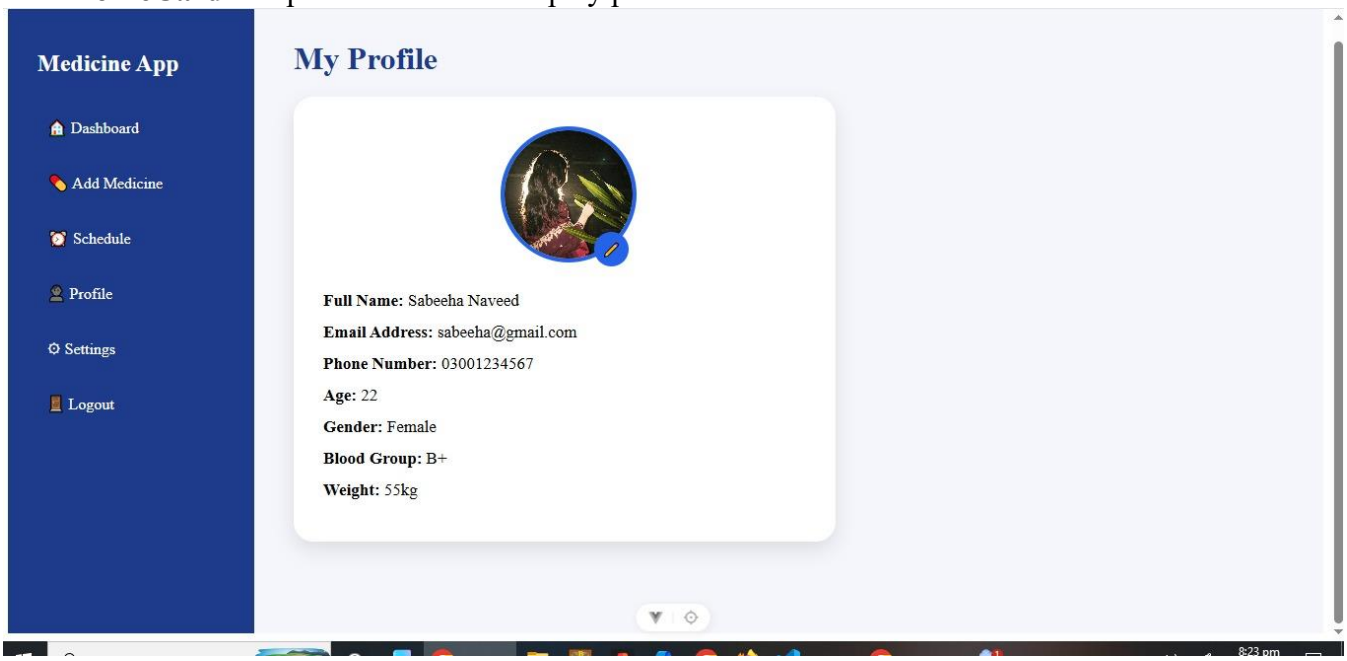
- Taken medicines
- Missed medicines
- Date and time history

7.8 Profile Page

The Profile page displays user information.
Information includes:

- Name
- Email
- Phone Number
- Age
- Gender
- Blood Group
- Weight

The **ProfileCard** component is used to display profile information.

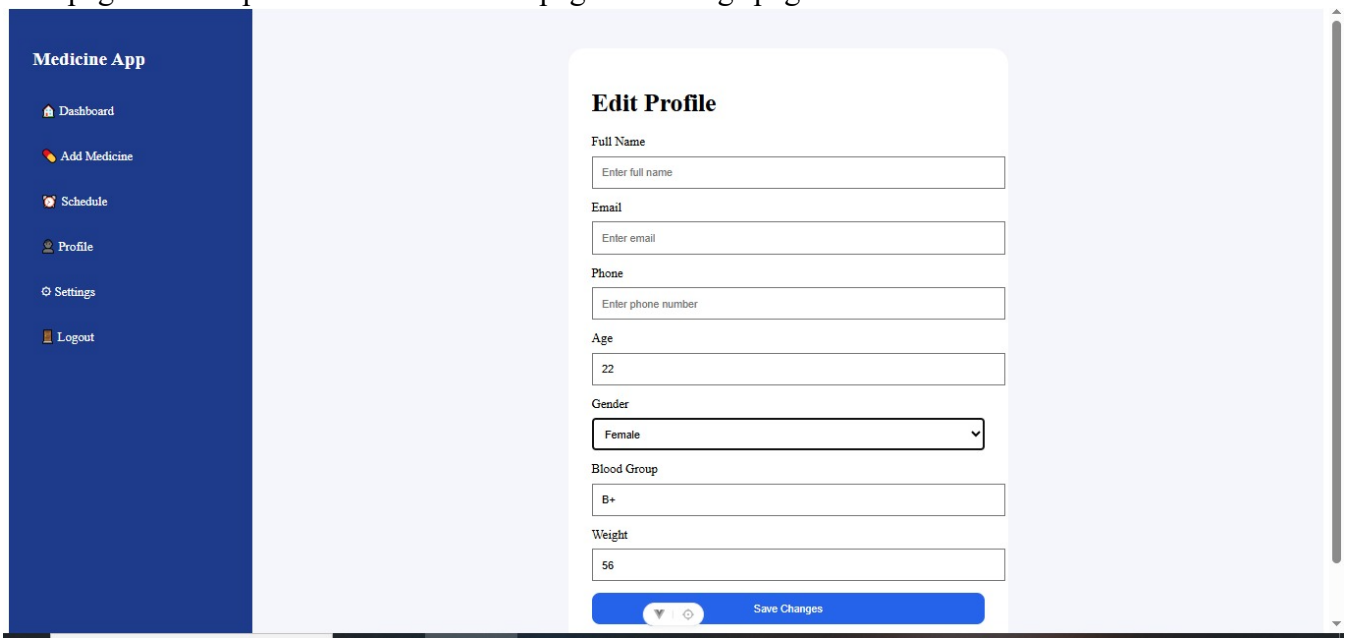


7.9 Edit Profile Page

This page allows users to update their profile information.
Users can edit:

- Name
- Phone Number
- Age
- Gender
- Blood Group
- Weight

This page can be opened from the Profile page or Settings page.



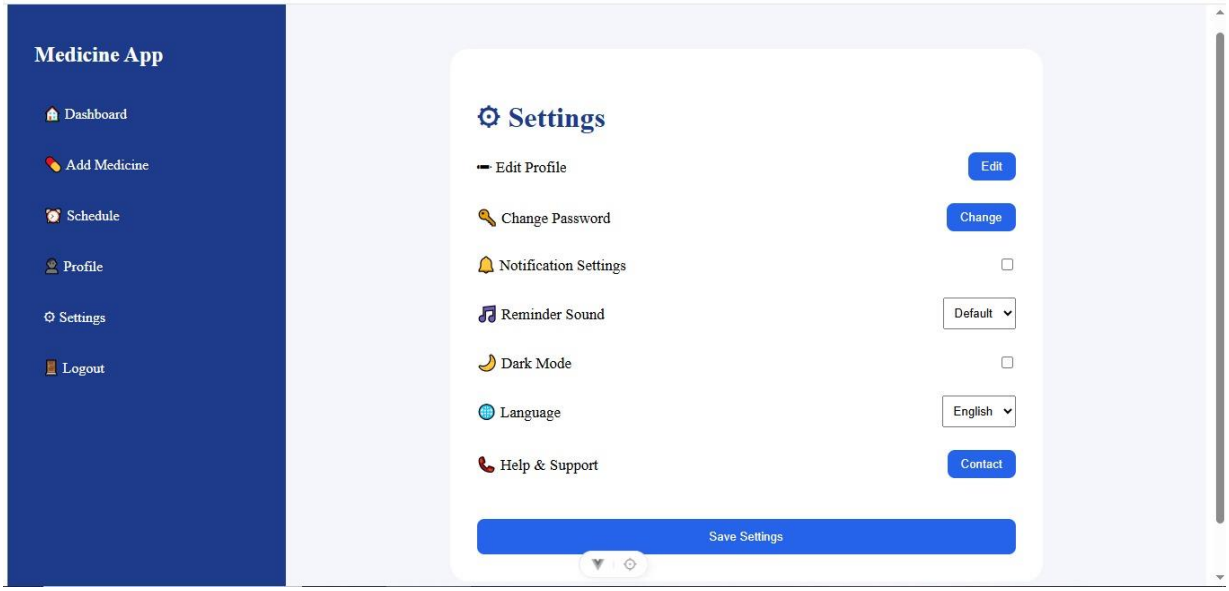
7.10 Settings Page

The Settings page allows users to customize the application.

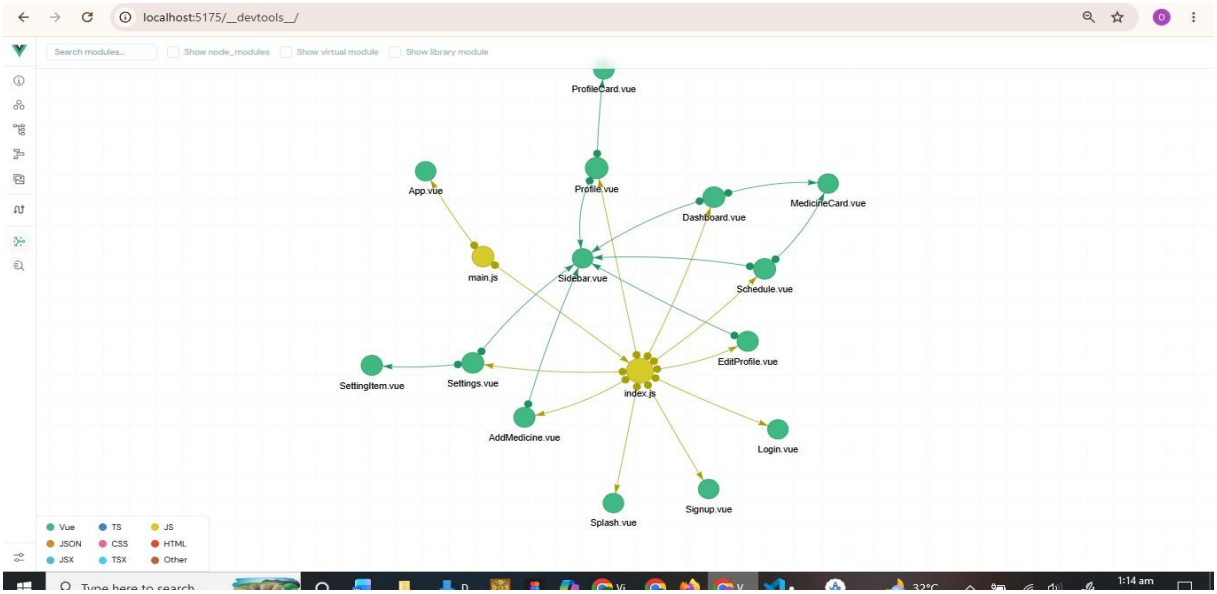
Features include:

- Edit Profile
- Change Password
- Notification Settings
- Reminder Sound
- Dark Mode
- Language
- Help & Support

The **SettingItem** component is used for each setting option.



8. Architecture Tree



9.Code Snippets

Props

Props are used to pass data from a parent component to a child component.
Parent → Child

MedicineCard props:

```
const props = defineProps({
  medicineName: String,
  medicineTime: String
})
```

ProfileCard props:

```
defineProps({
  name: String,
  email: String,
  phone: String,
  age: Number,
  gender: String,
  bloodGroup: String,
  weight: String
})
```

Setting props:

```
const props = defineProps({
  title: String,
  type: String,
  buttonText: String,
  link: String,
  options: Array
})
```

Sidebar props:

```
defineProps({
  title: String
})
```

Emitters

Emitters are used to send events from a child component back to the parent component.

Flow:

Child → Parent

MedicineCard Emitters:

```
const emit = defineEmits(['medicineTaken'])

const markTaken = () => {
  emit('medicineTaken', props.medicineName)
}
```

Profile Card Emitts:

```
const emit = defineEmits(['editClicked'])

const editProfile = () => {
  emit('editClicked')
}
```

Setting Emitter:

```
const router = useRouter()

const props = defineProps({
  title: String,
  type: String,
  buttonText: String,
  link: String,
  options: Array
})

const emit = defineEmits(['settingChanged'])

const handleClick = () => {
  emit('settingChanged')

  // popup force show
  alert("Setting updated successfully")

  // after popup close, navigate
  if (props.link) {
    router.push(props.link)
  }
}

const sendEvent = () => {
  emit('settingChanged')
  alert("Setting updated successfully")
}
```

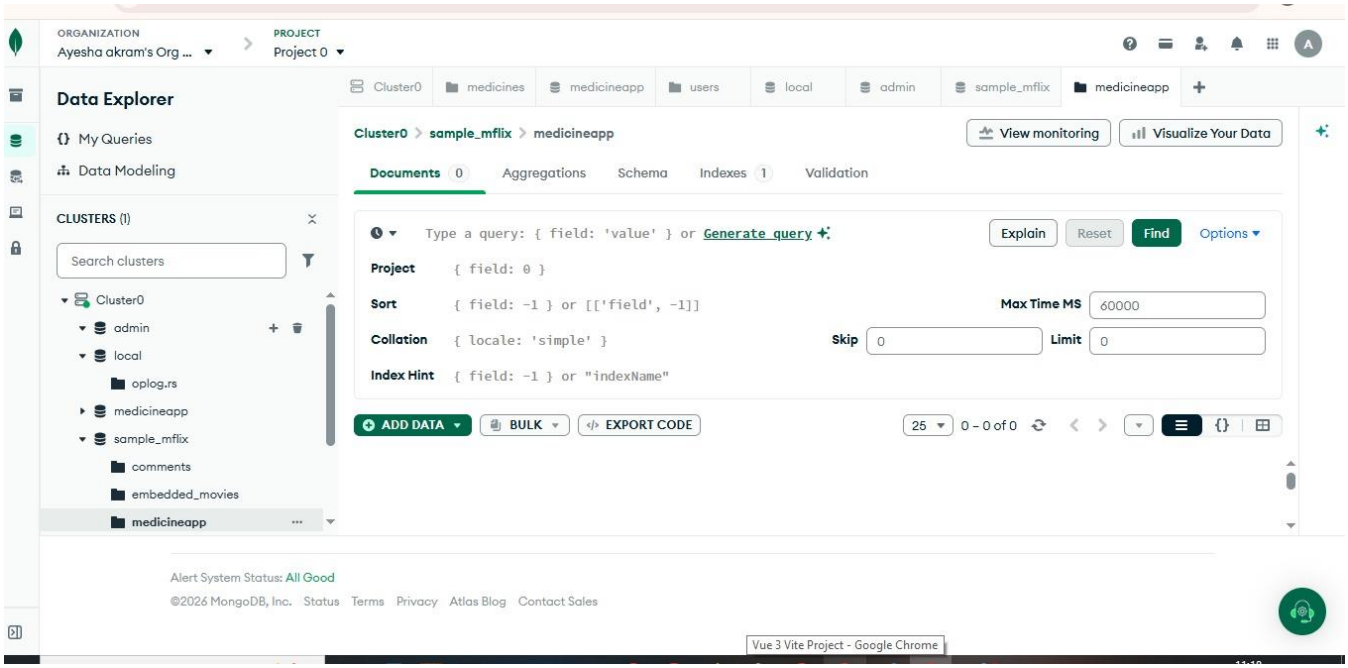
10. Server-Side Architecture Results

The backend provides:

- Secure Authentication
- Password Encryption
- JWT Authorization
- CRUD Operations
- MongoDB Integration
- REST APIs
- Secure Data Storage

11. Database Schema Design

The Medicine Reminder System uses MongoDB Atlas, a NoSQL cloud database, to store and manage application data. The database is designed using two main collections: Users and Medicines. The Users collection stores user information such as name, email, password, and profile details, while the Medicines collection stores medicine-related information including medicine name, dosage, reminder time, frequency, and treatment dates.



12. Conclusion

The Medicine Reminder System successfully provides a secure and user-friendly platform for managing medicine schedules. The application is built using Vue.js for the frontend and Node.js with Express.js for the backend. MongoDB Atlas stores user and medicine data securely. RESTful APIs enable communication between the frontend and backend. The modular architecture, authentication mechanisms, and cloud database make the application scalable, secure, and easy to maintain.

References

1. Vue.js Documentation – https://vuejs.org/
2. Node.js Documentation – https://nodejs.org/
3. Express.js Documentation – https://expressjs.com/
4. MongoDB Documentation – https://www.mongodb.com/docs/
5. Mongoose Documentation – https://mongoosejs.com/docs/
6. Axios Documentation – https://axios-http.com/
7. JWT Documentation – https://jwt.io/
8. MDN Web Docs – https://developer.mozilla.org/